

Raytracer Project Report

Léonard Mauvernay

Abstract

In my CSC_3S006_EP course, I built a path tracer over the first four lab sessions. It handles diffuse and mirror surfaces, hard shadows from a point light, indirect lighting with a fixed bounce limit, antialiasing, ray-triangle intersection on an OBJ mesh, and a bounding volume hierarchy. All equations and conventions follow the lecture notes [1]. I report below the renders, my measured render times, two short debugging notes, and a small set of performance experiments I ran while writing this report. All timings are on a MacBook Air M2.

Setup

All images are 512×512 . The camera sits at $Q = (0, 0, 55)$ and looks toward $-z$ with a 60° horizontal field of view. The point light is at $S = (-10, 20, 40)$. The room is built from six large spheres for the walls, ceiling, and floor, as on page 15 of [1]. Output is gamma corrected with $\gamma = 2.2$ and the pixel loop uses `#pragma omp parallel for schedule(dynamic, 1)`. The course provided `stb_image.h`, `stb_image_write.h`, the OBJ loader inside `TriangleMesh::readOBJ`, the cat OBJ model, and a skeleton `main.cpp`.

Lab 1: Diffuse and mirror surfaces, direct lighting and shadows

Primary rays and sphere intersection. For pixel (i, j) I build the camera ray direction as on page 15 of the lecture notes and normalize it. For a sphere of center C and radius R and a ray $O + tu$ with unit direction u , plugging into $\|O + tu - C\|^2 = R^2$ gives

$$t^2 + 2t\langle u, O - C \rangle + \|O - C\|^2 - R^2 = 0, \quad \Delta = \langle u, O - C \rangle^2 - (\|O - C\|^2 - R^2). \quad (1)$$

When $\Delta \geq 0$, the two roots are $t_{1,2} = -\langle u, O - C \rangle \mp \sqrt{\Delta}$ and I keep the smallest nonnegative one. The loop iterating over both roots in `Sphere::intersect` was suggested by Louis Legrain.

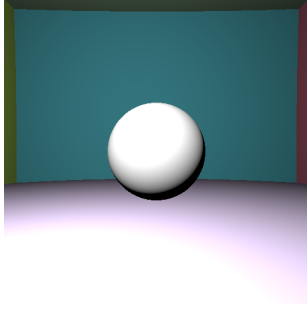
Diffuse shading. For a diffuse surface with albedo ρ , the radiance is

$$L = \frac{I}{4\pi d^2} \frac{\rho}{\pi} V_P(S) \max(0, \langle N, \omega_i \rangle), \quad \omega_i = \frac{S - P}{\|S - P\|}, \quad d = \|S - P\|. \quad (2)$$

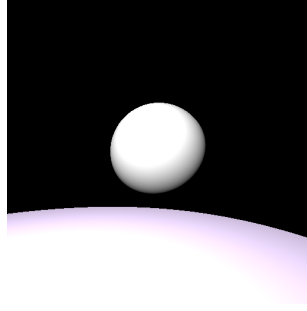
$V_P(S)$ is 1 when a ray from $P + \epsilon N$ in direction ω_i hits nothing or hits only past the light, and 0 otherwise. The offset $\epsilon = 10^{-5}$ was advised by Xianjin Gong.

Mirror. On a mirror hit I reflect the ray direction as $u_r = u - 2\langle u, N \rangle N$ and call `getColor` recursively in direction u_r . Recursion stops when the depth reaches `max_light_bounce`, set to 5.

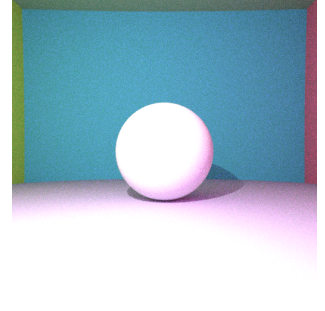
Bug during lab 1. My first shadow ray used the raw vector $S - P$ instead of the normalized direction. `Sphere::intersect` assumes a unit direction, so passing $S - P$ directly makes the test invalid. In my render this turned the background black, Figure 1b. The fix stores $d^2 = \|S - P\|^2$, normalizes the direction, then compares the squared distance to the shadow hit point against d^2 .



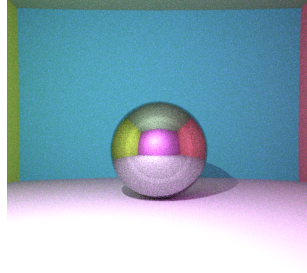
(a). Lab 1, direct lighting only, no shadow test, 0.049s



(b). Lab 1, shadow ray bug (not normalized), 0.048s



(c). Lab 2, fixed shadows, indirect, antialiasing, $N = 30$, 6.527s



(d). Mirror center sphere, $N = 30$, 6.180s

Figure 1. Lab 1 uses 1 ray per pixel. Lab 2 uses $N = 30$ paths per pixel with 5 bounces. The mirror render was produced by setting the center sphere's mirror flag to `true`.

Lab 2: Indirect lighting and antialiasing

Indirect lighting. On a diffuse hit the rendering equation simplifies to

$$L_o(x, \omega_o) = \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \langle \omega_i, N \rangle d\omega_i. \quad (3)$$

I sample ω_i proportionally to $\langle \omega_i, N \rangle / \pi$. In the local frame aligned with N , with $r_1, r_2 \sim \mathcal{U}(0, 1)$, a sample is given by

$$x = \cos(2\pi r_1) \sqrt{1 - r_2}, \quad y = \sin(2\pi r_1) \sqrt{1 - r_2}, \quad z = \sqrt{r_2}. \quad (4)$$

I build the local frame by finding the axis where N has the smallest absolute component, crossing that axis with N to get T_1 , then setting $T_2 = N \times T_1$. After importance sampling the cosine and $1/\pi$ cancel against the pdf, so the estimator reduces to $\rho \cdot L_i$. I make one recursive call per hit and sum the direct and indirect contributions. Recursion stops at `max_light_bounce`, which is here set to 5. I do not use Russian roulette.

Antialiasing. I sample each pixel $N = 30$ times with a Gaussian offset of $\sigma = 0.5$ pixel drawn by Box-Muller. Samples are averaged before gamma correction. Figure 1c shows the result.

Mirror. Setting the mirror flag on the center sphere produces Figure 1d. The reflection correctly shows the surrounding walls and floor. This confirms that the recursive mirror branch works.

Lab 3: Ray mesh intersection

The mesh is the cat OBJ from the course resources, with 3954 triangles, scaled by 0.6 and translated by $(0, -10, 0)$.

Möller-Trumbore. For a triangle A, B, C with $e_1 = B - A$, $e_2 = C - A$, and $N = e_1 \times e_2$ (not normalized), Cramer's rule applied to $\beta e_1 + \gamma e_2 - tu = O - A$ gives

$$\beta = \frac{\langle e_2, (A - O) \times u \rangle}{\langle u, N \rangle}, \quad \gamma = -\frac{\langle e_1, (A - O) \times u \rangle}{\langle u, N \rangle}, \quad \alpha = 1 - \beta - \gamma, \quad t = \frac{\langle A - O, N \rangle}{\langle u, N \rangle}. \quad (5)$$

A hit is accepted when $\alpha, \beta, \gamma \geq 0$ and $t \geq 0$. The professor suggested rejecting β or γ outside $[0, 1]$ with an early `continue`. I implemented this rejection in lab 4, before moving on BVH.

Mesh bounding box. I test the ray against the axis aligned bounding box of the whole mesh before testing any triangle. In my lab 3 code this box is recomputed inside `TriangleMesh::intersect` on every call. Per axis I compute $t_0^x = (B_{\min}^x - O^x)/u^x$ and $t_1^x = (B_{\max}^x - O^x)/u^x$, swap them when needed, and continue to the triangle loop only when $\max(t_0^x, t_0^y, t_0^z) \leq \min(t_1^x, t_1^y, t_1^z)$.

Bug fixed in lab 4. My lab 3 code used `dot(ray.u, N)` as the denominator, where `N` is the output normal parameter and not the current triangle normal. This made the Möller Trumbore formula invalid. The fix uses `dot(ray.u, N_temp)`, where `N_temp = cross(e1, e2)`. Figure 2 was rendered with the fixed version.

Lab 4: BVH and performance experiments

Each BVH node stores a contiguous range of triangle indices and a bounding box.

Listing 1. BVH node.

```
struct BVH_node {
    Vector b_min, b_max;
    int start_index, end_index;
    int left, right;
};
```

Build. `build_BVH(node, start, end)` computes the tight box of the triangles in `[start, end)`. If the range has at most 4 triangles, the node is a leaf. Otherwise, I split along the longest box axis. I discussed this axis choice with Louis Legrain. Triangles are partitioned by comparing each centroid with the midpoint on that axis. A degenerate split falls back to the median index so recursion always terminates. Two new nodes are appended to the array and the function recurses on both halves.

Traverse. The traversal tests the bounding box of the current node. If it is missed, the subtree is skipped. If it is hit and the node is internal, both children are visited recursively. At a leaf, all triangles in the range are tested with (5). The output t is initialized to $+\infty$ before the first call, so the test `t_temp < t` keeps the closest hit across the whole tree.

Experiments. I ran five configurations to measure the sources of speedup. The OpenMP run with shared `engine[0]` is diagnostic only. Indeed, multiple threads modify the same `engine` at once, which is a data race (page 32). Results are in Table 1.

Configuration	Time
Lab 3, single thread	381.637 s
Lab 3, OpenMP, shared <code>engine[0]</code> (diagnostic only)	79.127 s
Lab 3, OpenMP, per-thread engine	71.972 s
Lab 4, BVH, single thread	11.324 s
Lab 4, BVH, OpenMP, per-thread engine	2.477 s

Table 1. Render times for the cat scene. 512×512 , $N = 30$, and `max_light_bounce = 5`

OpenMP gives a large speedup because image rows are rendered in parallel. Replacing the shared engine with one per thread removes the data race in random sampling. The BVH gives the largest gain since most rays miss bounding boxes early and never reach the triangle loop.

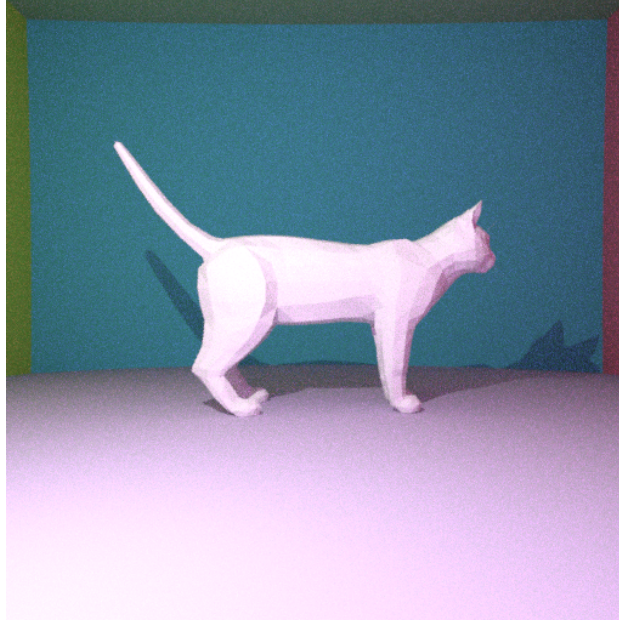


Figure 2. Cat mesh, BVH, $N = 30$, `max_light_bounce = 5`, albedo $(0.8, 0.8, 0.8)$, light intensity 1.5×10^7 . Render time 11.324 s

Compilation commands

The commands below were used for the timings in Table 1. The source file was updated for each lab version before compiling.

```
# Labs 1, 2, 3 (single thread), and 4 (single thread)
clang++ main.cpp -O3 -march=native -std=c++17 -o main
```

```
# Lab 3 and 4 with OpenMP on macOS
clang++ main.cpp -O3 -march=native -std=c++17 \
-Xpreprocessor -fopenmp \
-I"$(brew --prefix libomp)/include" \
-L"$(brew --prefix libomp)/lib" \
-lomp -Wno-deprecated-declarations -o main
```

The per-thread engine lab 4 file is in the GitHub repository and was written while producing this report.

Acknowledgments and external sources

I wrote all the code. Louis Legrain and I discussed parts of the implementation during the lab sessions, such as the longest axis logic in `build_BVH`. Xianjin Gong advised $\epsilon = 10^{-5}$ for the shadow offset. Professor Bonneel suggested the early `continue` in the triangle test. These inputs are noted in the source comments. I also used standard references such as `cppreference.com` and Stack Overflow. An AI assistant (ChatGPT) was used only to generate shell commands such as the OpenMP compilation command detailed in the compilation commands section.

References

- [1] N. Bonneel, *Computer Graphics, lecture notes*, <https://moodle.ip-paris.fr/mod/resource/view.php?id=181518>.